1. Introduction

A question answering system is the latest technology used in natural language processing that can allow people to input questions in natural language and provide accurate contextu-ally relevant answers. In other words, QA systems are helpful for a lot of purposes nowadays, like information retrieval or customer support services, where users access relevant information directly without needing to sift through huge piles of data. Though QA systems are widely applied for high-resource languages such as English and French, developing QA systems for low-resource languages is a challenging task. The main challenge is that there is a lack of annotated training data, a resource that forms the backbone of most modern NLP systems. In high-resource languages, extensive datasets are available, allowing for the development of robust QA models. However, for low-resource languages, structured digital re-sources are usually missing, and thus re-searchers and engineers must face a different land-scape with complexities in data acquisition and linguistic processing. For these languages, the development of an effective QA system is very important because it can bridge large gaps in information accessibility, enabling native speakers to retrieve valuable information in their own languages.

QA systems for all Dravidian languages, such as Tamil, Malayalam, Kannada, and Telugu, are in the very initial phases of development. Most researches are up to date for Tamil and Malayalam due to their rich user bases and digital prominence. Tamil and Malayalam have a rich morphological structure and agglutinative grammar where sentences could be con-structed by piling up morphemes on words. This morpho-logical richness translates to very extensive word forms that make it highly difficult for NLP models to tokenise, parse, and extract meanings. For example, the Malayalam word for 'education' (വിദ്യാഭ്യാസം) can take on multiple forms depending on context, with each suffix or prefix adding additional meaning or grammatical nuance. Moreover, the Dravidian languages possess distinctive syntactic features, like free word order and context-dependent use of particles, which are not characteristic of the English language. Such properties require sophisticated linguistic processing methods to obtain correct QA since word meanings and relations change according to inflectional and derivational morphology. This, in turn, means that there is not a lot of digitized content available for these languages that are mostly spoken in regional contexts, and there is a scarcity of linguistic resources, like full-scale lexicons and annotated corpora. There-fore, the challenge of building a good QA system for Malayalam and Tamil, as well as other Dravidian languages, is both technically and linguistically challenging.

The current state-of-the-art approaches in QA for Malayalam and Tamil leverage various neural architectures, including transformers, sequence-to-sequence models, and embeddings are derived from multilingual pre-trained models such as mBERT, IndicBERT, and XLM-R. These models represent significant improvements in NLP for low-resource languages, but they rely heavily on transfer learning from high-resource languages. This approach often faces limitations because multilingual pre-trained models are, for the most part trained on larger languages and may lack the necessary representation for Dravidian languages, which are highly morphologically and syntactically distinct. For example, while mBERT and IndicBERT include some Indian languages, their ability to model complex syntax and morphology in Malayalam and Tamil is limited, especially when faced with context-dependent particles unique to Dravidian languages.

A few papers have been done on leveraging transformer-based models fine-tuned on Tamil datasets but are largely preliminary due to the limitations of data. Like these, similar efforts in the case of Malayalam showed a high success rate adapting the model pre-trained in any other language with cross-lingual embeddings but limits its accuracy and fluency. Moreover, there are far fewer resources for Kannada and Telugu QA tasks; consequently, most approaches for these languages are still experimental or very much limited to small datasets. Kannada, for example, has even fewer available linguistic datasets than Tamil or Malaya-lam, which makes applying pre-trained LLMs really quite difficult without extensive adaptation. In reality, other Dravidian languages that have little or no digital representation face an even greater gap, leaving QA for the most part unexplored in NLP research. It's worth noting that with large language models like GPT-3, BERT, and T5 being reported as showing promising results for all types of QA tasks with their deep contextual understanding ability, they can generate coherent and accurate responses. Their applicability to Dravidian languages is, however limited, by the requirement of large amounts of annotated datasets for the models which is very difficult to source for the latter.

Large Language Models have become the transformative approach to QA, offering capabilities much more sophisticated than those provided by the traditional models, as these models can comprehend very complex queries and provide answers. They are the core of most of the latest QA systems due to contextual reasoning, their internal vast knowledge, and the adaptation to any other language or domain. Using such techniques as prompting, retrieval-augmented generation (RAG), fine-tuning, and instruction tuning, LLMs have dramatically become better at dealing with QA tasks with contextual precision. Prompts can improve performance for the models by asking a query that best retrieves an appropriate answer so that the model may change or modify in response to user need according to type of question - a simple fact-based one to multi-layered complex questions.

RAG is another key innovation that made LLMs' performance in QA systems stronger than ever. It helps extract relevant contextual data by supplementing the internal knowledge with external sources of information; this is helpful in low-resource languages, where internal knowledge may be too limited due to the lack of proper training data. RAG's retrieval mechanism accesses external information sources, which may be documents or databases. This enables the model to provide enriched responses based on both pre-trained knowledge and newly retrieved content, making it very adaptive to complex QA scenarios. Fine-tuning further enhances LLMs' performance in specific QA tasks by training the models on curated datasets, allowing them to understand specialized language patterns, domain-specific terminologies, and even user-specific preferences. Such processing is important to tuning up models for the adaptation for Dravidian languages by fine-tuning model's parameters on specific characteristics in syntax or even unique inflection forms particular to Malayalam or Tamil- for instance verb-subject-object structure or certain special kinds of inflectional.

Instruction-tuning, a specialized form of fine-tuning, has expanded the adaptability of LLMs by training them to follow specific instructions or prompts accurately, even for intricate and multi-step questions. By aligning the model’s responses to task objectives, instruction-tuning is especially beneficial for QA tasks in linguistically diverse contexts, where language-specific rules and cultural nuances play a significant role in interpretation. This approach to tuning guarantees structured, on-topic answers; in particular, in highly complex syntactic constructions of the type found in Malayalam or Tamil, answer formats are more difficult to acquire

accurately. With these high-level methodologies, LLMs have unlocked new frontiers for QA applications, with scarce resources in some language scenarios. But, on the other hand, such approaches also reveal the computational complexity and the requirement of da-ta, which might be a hurdle in low-resource environments for languages like Tamil and Malayalam.

Further complicating the application of LLMs to QA in Malayalam and Tamil are practical deployment challenges, especially in resource-constrained settings where computational and memory resources are limited. Although the powerful models like GPT-3, BERT, and T5 have successfully been applied in high-resource environments, their application to Dravidian languages raises issues of scalability and accessibility. Many QA models for Malayalam and Tamil, among others, must run within very constrained device capabilities such as mobile phones or edge devices, especially when reaching the rural and remote areas in which these languages are spoken. LLMs require a lot of processing power, which complicates their deployment on edge devices, where low latency and efficiency are critical for practical usability. For example, deploying GPT-3 or BERT on a normal mobile device would consume all the battery power within a very short period and would also cause slow response times, which is a drawback in real-time QA applications.

Moreover, most instruction-tuned LLMs, which are critical for QA in Tamil and Malayalam because they can interpret very complex commands, are engineered with high-resource languages, which require large amounts of data and significant computational resources. However, the high-quality instruction data needed for such models is particularly scarce for Dravidian languages. Instruction-tuned models typically depend on large, curated datasets in which the model is trained to respond to a diverse set of question types and subtle prompts, a resource which is very far from what is needed for Malayalam or Tamil. This lack of instruction data severely limits the ability of QA models to correctly interpret and answer questions in these languages, particularly those requiring context or cultural knowledge.

There is a pressing need for QA models that can operate effectively in low-resource, low-power settings without compromising on accuracy. In response to this need, research has increasingly turned to “tiny LLMs”—compact versions of traditional large language models—designed to retain the critical benefits of LLMs but with reduced model sizes and computational demands. These smaller models allow for the efficient delivery of QA services for Malayalam, Tamil, and other Dravidian languages without the need for heavy infrastructure. Although tiny LLMs lack the extensive parameter count of their larger counter-parts, they have shown competitive performance in handling domain-specific tasks, especially when fine-tuned with the right strategies and augmented with external retrieval mechanisms like RAG. The combination of tiny LLMs with techniques such as LoRA (Low-Rank Adaptation), QLoRA, and QA-LoRA further enhances these models' adaptability, allowing the models to focus on relevant information for the task in question without overloading re-sources. Tiny LLMs therefore offer a promising path to developing accessible, trustworthy QA systems that meet the needs of Dravidian language speakers in low-resource environments.

To overcome these challenges, we propose the use of tiny LLMs, compact versions of traditional LLMs, that are optimized for low-resource settings using strategies such as LoRA, QLoRA, and QA-LoRA. Tiny LLMs present a balanced approach toward QA for Malayalam, Tamil, and other Dravidian languages, as it uses fewer computational resources while ensuring competitive accuracy. LoRA, QLoRA, and QA-LoRA allows such models to cap-ture the

specific linguistic patterns in a resource-efficient manner by focusing the fine-tuning only on the most impactful parts of the model. QA-LoRA technique is engineered to-wards QA-specific tasks allowing this model to focus on understanding contexts and accurate responses for the languages from the Dravidian branch. But allowing the QA system to also retrieve relevant external information from incorporating Retrieval-Augmented Generation (RAG) into its model pipeline adds depth and relevance to its responses. Our experiments have shown that these compact LLMs can handle complex queries and thus demonstrate that tiny LLMs are a viable scalable solution for building QA systems suitable to the needs of speakers of low-resource languages in resource-constrained environments.

The structure of this chapter is as follows: The first section introduces a com-prehensive overview of related works, including the current approaches toward QA in low-resource languages and more particularly in Dravidian languages, focusing on their limitations. This is then followed by a detailed description of the dataset, including a discussion on the IndicQA dataset used in our fine-tuning process and how it was selected and prepared to suit QA tasks for Malayalam, Tamil, and other Dravidian languages. In the methodology section, we delve into the techniques employed to fine-tune tiny LLMs, specifically focusing on LoRA, QLoRA , and retrieval-augmented generation as key components. Next come experiments and results to give a rich evaluation of the model in terms of performance, in terms of assessing accuracy, relevance, and computation efficiency, for QA about Dravidian languages. Finally, we conclude by summarizing findings, implicating results for future system development, and giving pointers for directions of future re-search before coming to references.

2. Related Works

Malayalam, as a richly morphologically structured language and agglutinative in nature, creates immense challenges toward developing effective QA systems. A wide variety of deep learning methods have been proposed for overcoming the above complexities. The hybrid framework combining CNN and Bi-LSTM networks is reported to effectively capture se-mantic and contextual features at a classification accuracy of more than 91% [1]. Similarly, Memory Network models, along with LSTM and GRU, have performed well, and the Memory Networks have outperformed the others at 80% accuracy, which shows it is feasible to handle complex questions in Malayalam [2]. These are backed by the datasets tailored for Malayalam QA. The Facebook bAbI dataset, famous for tasks that demand multi-step reasoning, has been adapted to Malayalam too, giving a benchmark to scalable QA models [1][2]. Furthermore, the annotated SQuAD dataset has been used for domain-specific applications like health-related QA where the fine-tuned BERT models have shown an F1 score of 86% [3]. To address the issue of the scarcity of translation data, techniques of data augmentation also include back-translation as well as synthetic text generation [4]. These latter techniques have particularly been instrumental in improving the performance of code-switching between Malayalam and English in a conversational setting [5]. In health-related QA, semantic systems using word embeddings and cosine similarity have shown to be effective; transformers like BERT, further fine-tuned with neural word embeddings, have gone on to improve the accuracies of responses [3]. These developments notwithstanding, Malaya-lam remains linguistically challenging and suffers from the dearth of annotated datasets; thus, the demand for creative deep learning techniques and practical data augmentation methods will only increase QA systems for this language.

Tamil QA systems have progressed significantly in deep learning and machine learning techniques to overcome the complexity of the Tamil language. One such approach is developing QA systems using deep learning models, which process questions through interfaces, retrieve relevant contexts, and generate responses with high accuracy, validated using manually created test datasets [6]. The XLM-RoBERTa language model, trained on both Tamil-exclusive and mixed-language datasets, has also demonstrated promising results, with robust performance when evaluated against hand-annotated data [7]. For domain-specific applications, systems employing Hidden Markov Models (HMM) with the Viterbi algorithm for parts-of-speech tagging have been implemented, using locality-sensitive hashing for document retrieval and Jaccard similarity for response generation. These systems have proved to be highly successful in ad-dressing agriculture-related queries [8]. Further research into automatic question and answer generation has also been achieved with the rule-based pro-cessing technique and the named entity recognition (NER) techniques to achieve an accuracy of 62.22% on grammatically correct and meaningful questions on historical domain [9]. For NER, the Bi-LSTM network was found to be very useful, capturing n-gram patterns and syntactic features with character-based and word2vec embeddings that could reach high ac-curacy and F-scores on Tamil datasets [10]. The diverse methodologies clearly mark the rap-id advancement of the QA systems in Tamil with regards to overcoming linguistic problems and extending applicability to numerous domains.

An interested application of LLM for question answering in the case of Dravidian languages especially Malayalam and Tamil language is because it brings under study linguistic diversity and complexity at which those languages operate. New tests evaluate the multilinguistics competency that models for quality by benchmarking, MULTIQ, is another benchmarking which tries testing, even questions set for the accuracy in multilingual QA on variety of tests [11]. Shared tasks and workshops such as Dravidian-LangTech have immensely contributed toward advancing machine translation and multi-task learning for the Dravidian languages to allow LLMs to perform well on code-mixed and low-resource scenarios [12][13]. The same LLMs have been utilized for sentiment analysis and identifying offensive languages in Tamil and Malayalam. For these languages, competitive results are obtained from the code-mixed text datasets, reaching F1 scores of up to 0.96 for detecting offensive languages [14][15]. Further, a deep learning-based QA system for Malayalam, including Long Short-Term Memory Networks and Memory Networks, has been found to be highly accurate in ex-tracting exact answers [1].

For Tamil QA, fine-tuning multilingual LLMs such as mBERT and XLM-RoBERTa on domain-specific datasets has shown improved performance in handling Tamil-exclusive and mixed-language inputs, validating their utility for low-resource languages [16]. Further efforts within domain-specific QA systems include health-related applications, which employ NER and transformer-based architectures to improve performance metrics in Tamil [17]. These advances highlight the revolutionary value of LLMs within QA systems for Dravidian languages, while further exploitation and fine-tuning shall be necessary to address those subtle linguistic challenges and augment domain-specific adaptability in general.

Research on using tiny LLMs for QA in Dravidian languages such as Malayalam and Tamil is at a very initial stage. However, other related work in NLP, including fine-tuning, trans-former models, and retrieval-augmented approaches, provides guidance for future work in the area of QA for low-resource languages. One such work is the detection of code-mixed offensive

content in Tamil, Kannada, and Malayalam. The model fine-tuned in a bidirectional manner achieves an F1-score of 0.96 for Malayalam and 0.73 for Tamil. This proves the effectiveness of fine-tuning on social media data that are prone to code-mixing [18]. In language modelling in Tamil, morpheme-based approaches combined with Katz back-off smoothing led to significantly better speech recognition than word-based models [19].

Further developments in machine translation for Dravidian languages, such as Tamil, Malayalam, and Telugu, have also laid a strong foundation for NLP tasks. Pre-processing methods like Byte Pair Encoding (BPE) and shared tasks involving over 40 research groups have improved BLEU scores, emphasizing the importance of robust pre-training for low-resource languages [20], [23]. Cross-lingual learning has also been tried for pronoun resolution in which knowledge gathered from Tamil was used for improving the performance of the Malayalam model using CRFs [21]. Other works include the sentiment analysis code-mixed Tamil-English and Malayalam-English text wherein lightweight models, such as fastText outperformed more resource intensive models with F1-scores of 0.63 for Malayalam-English and 0.58 for Tamil-English [22].

Transformer models have also gone well with Dravidian languages. Neural Machine Translation for Kannada into Tamil, Telugu, and Malayalam used Seq2Seq with transformer architecture and fine-tuning achieved high BLEU scores [24]. In detecting misinformation, a transformer-based Malayalam-BERT model performed better in detecting fake news by achieving a macro F1 score of 0.88 and also ranked first in a competition task [25]. While these works do not explicitly address QA systems using tiny LLMs, they high-light techniques such as fine-tuning, domain adaptation, and cross-lingual learning, which are essential for adapting LLMs to low-resource settings like Malayalam and Tamil QA tasks.

Dravidian language systems like Malayalam and Tamil, as applied in research for QA systems, indicate that linguistic complexity represents a significant challenge for morphological richness and code-mixing. Hybrids from CNN-BiLSTM and Memory Networks promise new levels of deep learning architectures and achieve up to more than 90% accuracy for the Malayalam QA system. Tamil: Approaches based on fine-tuned multilingual LLMs, including XLM-RoBERTa and transformer-based architecture, have shown to provide strong performance across various applications that are domain-specific, for instance, health and agriculture. Data augmentation techniques used are back-translation, and synthetic text generation with respect to the scarcity of annotated datasets. Innovations in neural machine translation, and sentiment analysis underline that the transformer models are adaptable. Despite these facts, the need for a lightweight, resource-efficient model suitable for low-resource settings remains high. Fine-tuning, retrieval-augmented generation, and cross-lingual learning seem to enhance tiny LLMs for the advancement of QA systems in Dravidian languages with better scalability and linguistic adaptability.

No.	Paper	Focus Area	Key Contributions
[1]	Malayalam Question Answering System Based on a Deep Learning Hybrid Model of CNN and Bi-LSTM Approach	Hybrid Models for QA	Achieved 91% accuracy using CNN and Bi-LSTM to capture semantic and contextual features.
[2]	Malayalam Question Answering System Using Deep Learning Approaches	Memory Networks	Outperformed LSTM and GRU with 80% accuracy for complex queries; adapted Facebook bAbI dataset.
[3]	Neural Word Embedding-Based Transformer Model for Improving Malayalam Question Answering on Health Domain.	Domain-Specific QA	Fine-tuned BERT achieved 86% F1 score for health-related QA using an annotated SQuAD dataset.
[4], [5]	Data Augmentation Strategies for Low Resource Conversational Code-Switching, Data Augmentation for Code-Switch Language Modeling by Fusing Multiple Text Generation Methods	Data Augmentation	Back-translation and synthetic text generation improved QA performance in code-mixed scenarios.
[6]	Question Answering System for Tamil Using Deep Learning	QA system framework	High accuracy with deep learning-based QA frameworks validated using manually created datasets.
[7]	An Extractive Question Answering System for the Tamil Language	Multilingual Language Models	XLM-RoBERTa showed robust performance on Tamil-exclusive and mixed-language datasets.
[8]	Tamil question answering system using machine learning	Domain-Specific QA	HMM and Viterbi algorithm used for POS tagging in agriculture QA with high accuracy.
[9]	Domain specific Question & Answer generation in Tamil	Rule-based QA	Achieved 62.22% accuracy in generating grammatically correct questions in historical QA tasks.
[10]	Named Entity Recognition of Indian languages using Bi-LSTM	Named Entity Recognition	Bi-LSTM with word2vec embeddings improved F-scores in NER tasks.
[11]	Evaluating the Elementary Multilingual Capabilities of Large Language Models with MULTIQ	LLM Evaluation	MULTIQ benchmark assessed multilingual LLM performance in Malayalam and Tamil QA tasks.
[12], [13]	Neural Machine Translation On Dravidian Languages, Findings of the Shared Task on	Shared Tasks	DravidianLangTech workshops advanced machine translation and multitask learning for Dravidian languages.

No.	Paper	Focus Area	Key Contributions
	Multi-task Learning in Dravidian Languages		
[14], [15]	Overview of the DravidianCodeMix 2021 Shared Task on Sentiment Detection in Tamil, Malayalam, and Kannada, Offensive language identification in Dravidian code-mixed YouTube Comments and Posts	Sentiment Analysis	Competitive F1-scores for code-mixed Tamil-English and Malayalam-English sentiment analysis tasks.
[16], [17]	An Extractive Question Answering System for the Tamil Language, Neural Word Embedding-Based Transformer Model for Improving Tamil Question Answering on Health Domain.	Fine-Tuned LLMs	Improved performance for Tamil-exclusive and domain-specific QA tasks using mBERT and XLM-RoBERTa.
[18]	Offensive language identification in Dravidian code-mixed YouTube Comments and Posts	Code-Mixed NLP	Achieved F1-scores of 0.96 (Malayalam) and 0.73 (Tamil) for code-mixed offensive content detection.
[19]	Comparison of performance of enhanced morpheme-based language model with different word-based language models for improving the performance of Tamil speech recognition system	Morpheme-based Language Modeling	Improved speech recognition using Katz back-off smoothing with morpheme-based models.
[20], [23]	Neural Machine Translation, Resolving pronouns for a resource-poor language, Malayalam using resource-rich language, Tamil	Machine Translation	BPE pre-processing enhanced BLEU scores in Dravidian language translations.
[21]	Sentiment analysis for dravidian languages in code-mixed text	Cross-Lingual Learning	Tamil resources enhanced Malayalam performance in pronoun resolution using CRFs.
[22]	Findings of the Shared Task on Machine Translation in Dravidian languages	Lightweight Models	FastText outperformed resource-intensive models in code-mixed sentiment analysis.

No.	Paper	Focus Area	Key Contributions
[24]	Neural Machine Translation On Dravidian Languages	Neural Machine Translation	Seq2Seq and transformer models yielded high BLEU scores for Kannada into Tamil, Telugu, and Malayalam.
[25]	Fake News Classification Using Malayalam-BERT	Misinformation Detection	Transformer-based Malayalam-BERT achieved a macro F1 score of 0.88 in fake news detection.

Table1. Summary of Key Contributions in Dravidian QA

3. Dataset Description

The IndicQA dataset is a comprehensive resource to be used in developing and testing Question Answering systems for Indic languages. It accompanies each question with relevant context and answers, which will come in handy in supporting the conduct of research in low-resource Indic languages by making high-quality, structured data available. This dataset bridges an essential gap in NLP in the low-resource language group, especially in the Indic linguistic landscape.

3.1 Dataset Overview

The main features in the IndicQA dataset are:

•ID: ID for identifying each instance of the data.

•Context: Text that will provide all information required to answer the given question.

•Question: Query by the user that needs to be answered by the QA system.

•Answers: One or more valid responses to the question, which present various interpretations arising from the context.

It holds data for ten Indic languages, which represents the different linguistic and morphological patterns of the Indian subcontinent. We focus here on the datasets of Malayalam (ml) and Tamil (ta), given their linguistic richness and importance in the Dravidian family.

3.2 Languages of Focus

Malayalam and Tamil are selected subsets where significant speakers and linguistic barriers were the reason for doing so:

- Malayalam is morphologically rich and agglutinative. Thus, they will present some challenge regarding their inflectional complex pattern and the wide lexical scope of their vocabulary.
- Tamil has an old literary tradition also made up of agglutinative structures, special syntactic rules, and a very high degree of word variation, that would require strong NLP techniques.

3.3 Dataset Composition

The Malayalam and Tamil subsets of the IndicQA dataset are comprised of

- Context diversity: The context passages are drawn from different domains to ensure that the dataset captures a wide range of topics and thus makes it more versatile for real-world QA applications.
- Natural language queries: The questions reflect linguistic usage, regional variations and idiomatic expressions, varying phrases.
- Multiple correct answers: For most questions, there are multiple correct answers to allow for different ways of interpreting the question in light of the context given.

3.4 Dataset Statistics

The table below summarizes the statistics for the Malayalam and Tamil subsets of the IndicQA dataset:

Language	Malayalam	Tamil
Total Paragraphs	247	253
Total Questions	1589	1804
Avg. Questions/Paragraph	6.43	7.13
Total Words	61,977	63,150
Unique Words	30,534	27,331
Total Sentences	6725	5392
Avg. Sentence Length	9.22	11.71

Table2. Dataset Statistics

3.4 Utility of the Dataset

The diversity in paragraph contexts, natural language questions, and multiple valid answers in the IndicQA dataset makes it well-suited for evaluating QA systems. Its structured format provides a robust foundation for:

1. Training and fine-tuning tiny LLMs tailored for Malayalam and Tamil.
2. Addressing linguistic challenges posed by Dravidian languages, such as morphological richness and agglutinative structures.
3. Testing retrieval-augmented and fine-tuning techniques on real-world QA tasks.

By leveraging this dataset, we ensure that our QA models are exposed to realistic language usage scenarios, enabling better contextual understanding and response generation.

4. Methodology

In this study, three tiny LLMs from Hugging Face, as presented in Fig1—**Gemma 2B, Open Llama 3.1B,** and **T5 Base**—are used to design efficient QA systems for Dravidian languages. Given the 2 billion parameters of **Gemma 2B**, it is an ideal size balancing compactness and performance in low-resource tasks. It excels in understanding nuanced linguistic patterns and context while remaining computationally efficient, which is crucial for Malayalam and Tamil QA systems. **Open Llama 3.1B**, with 3.1 billion parameters, is optimized for multilingual tasks, enabling robust handling of linguistic diversity and code-mixed queries. Its enhanced contextual understanding makes it effective for precise answer extraction in diverse QA

scenarios. **T5 Base** with about 220 million parameters is a highly versatile text-to-text transformer that also excels at question-answering, summarization, and translation. It has a smaller size, meaning it's going to do fine-tuning and inference relatively faster. It is very ideal for real-time QA applications. Leveraging the unique strengths of these models, together with fine-tuning techniques and methodologies like Retrieval-Augmented Generation (RAG), this study explores the potential of these models in addressing the linguistic complexities of Malayalam and Tamil and operating within resource-constrained environments.

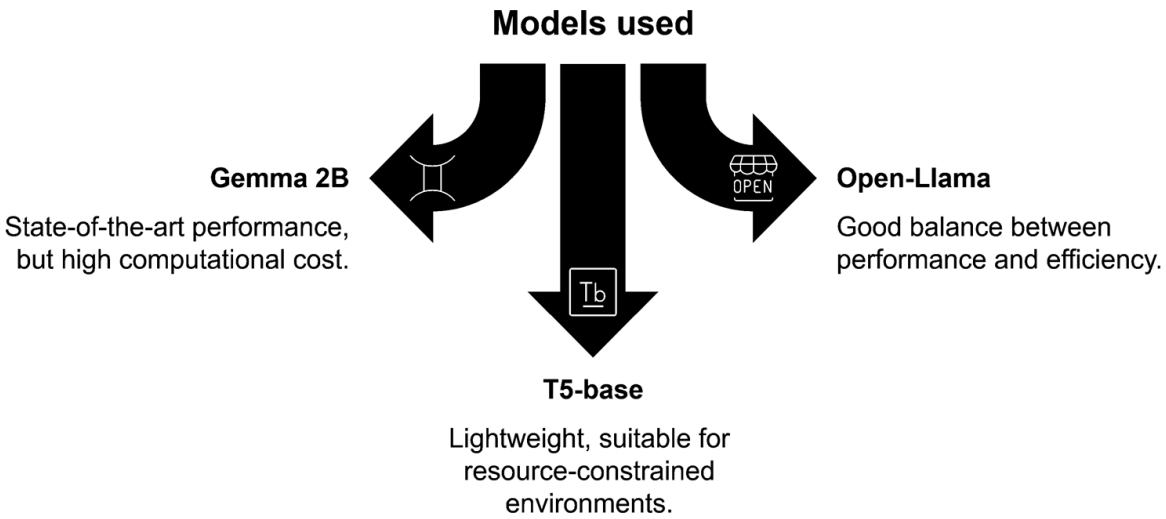

Fig1. Tiny LLMs used

This study employs a systematic approach to develop and evaluate QA systems for Dravidian languages using three tiny LLMs—Gemma 2B, Open Llama 3.1B, and T5 Base. The methodology involves the following steps and in fig2:

1. **Data Loading**
   The dataset consisting of question-answer pairs in Malayalam and Tamil is loaded into a suitable framework for processing. The data is structured to include contexts, questions, and answers to make sure that it is compatible with the input format required by the models.

2. **Data Pre-processing**
   Pre-processing involves cleaning and standardization of the data for improved model performance. This involves noise removal, tokenization, special characters, and normalization of code-mixed text. Language-specific tokenization strategies are used for multilingual support, in order to maintain linguistic flavor.

3. **Prompting (Zero/Few-Shot)**
   Initial experiments employ zero-shot and few-shot prompting to test the baseline capability of the models in question-answering performance without task-specific fine-tuning. Care is taken in crafting the prompts such that they steer the model toward coherent and contextually relevant answers.

4. **Retrieval-Augmented Generation (RAG)**
   RAG improves model performance by augmenting LLMs using knowledge retrievals from an external environment. This incorporates the pairing of LLMs with a retriever

module so that the system could retrieve appropriate context or documents while generating responses that are accurate and highly contextualized.

5. **Fine-Tuning Tiny LLMs with LoRA and QLoRA**
   Fine-tuning is applied to all three small LLMs with the methods LoRA (Low-Rank Adapta-tion) and QLoRA (Quantized Low-Rank Adaptation). Such techniques are computationally lightweight and allow the models to fine-tune for domain-specific QA tasks without significant memory usage.

6. **Generating Responses**
   After fine-tuning, all the models are asked to generate responses to a set of test questions. The generated responses will be judged for accuracy, coherence, and relevance against the context given.

7. **Evaluating Model Performance**
   ROUGE, Cosine Simi-larity, F1-score, and BLEU metrics are used to evaluate the performance of the models. These metrics test the correctness and fluency of the generated answers. Comparative analysis is carried out to identify the strengths and weaknesses of each model in dealing with the linguistic complexities of Malayalam and Tamil.

This multi-step methodology ensures the comprehensive evaluation of the tiny LLMs while addressing challenges in the resource-constrained environment of the Dravidian languages. Combining techniques of prompting, RAG, and fine-tuning demonstrate the potential of tiny LLMs in the delivery of effective QA systems for low-resource languages.

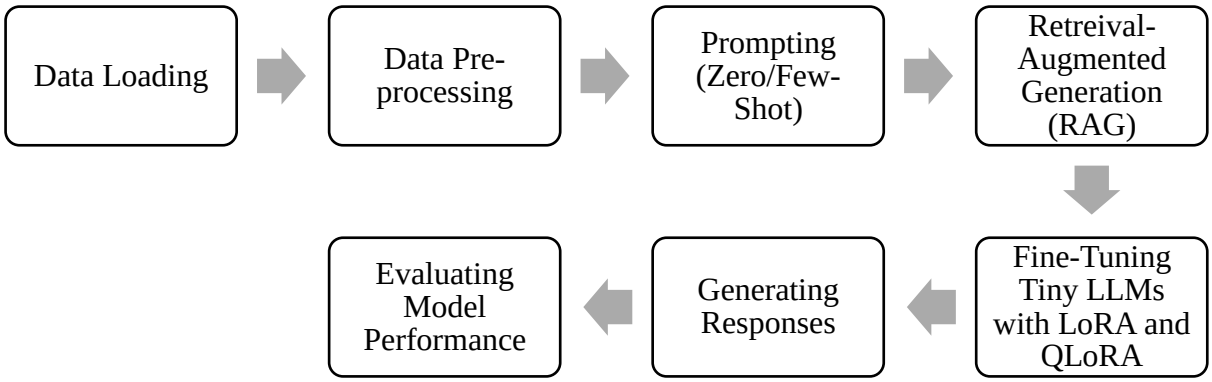

Fig2. Workflow of QA System Methodology

4.1 Approaches used

4.1.1 Prompting

A primary approach in using the language models to solve problems is prompting without changing its architecture explicitly. In terms of this paper, to induce desired responses from small LLMs, special input queries (prompts) were de-signed. However, this method is pretty advantageous when used in resource- con-strained environments and is devoid of the burden of large-scale fine tuning but guides the model towards certain behavior by context-rich and well-crafted examples.

Prompting uses the prior knowledge of language models and can be used for performing different tasks, such as question answering, summarization, or translation di-rectly through text instructions. In this work, both zero-shot prompting and few-shot prompting are used to study the baseline and improved abilities of the models in an-swering questions about Dravidian languages.

4.1.1.1 Zero-Shot Prompting

Zero-shot prompting involves testing the capacity of a model to perform some task without exposure to examples or explicit, task-specific instructions. The prompt directly formulates a query, thus relying only on the model's pre-trained knowledge in order to respond. This is particularly useful for assessing a model's general understanding and ability to adapt to novel tasks.

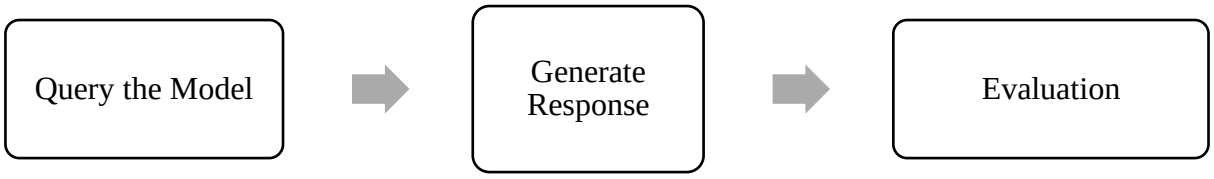

Fig3. Zero-Shot Prompting

Example (Malayalam):

- Question: ഹാരപ്പൻ കോട്ടയുടെ തെക്ക് ഭാഗത്ത് കണ്ടെത്തിയ പെയിന്റിംഗുകൾ, ഇത് ഹരപ്പൻ ജീവിതരീതിയിലേക്കുള്ള ആരുടെ വരവിനെ സൂചിപ്പിക്കുന്നു?
- Generated Answer: ഹാരപ്പൻ കോട്ടയുടെ തെക്ക് ഭാഗത്ത് കണ്ടെത്തിയ പെയിന്റിംഗുകൾ, ഹരപ്പൻ ജീവിതരീതിയിലേക്കുള്ള ആരുടെ വരവിനെ സൂചിപ്പിക്കുന്നു.
- Actual Answer: പരദേശിജീവിതരീതിയുടെ.

Example (Tamil):

- Question: வானியல், வானியற்பியல் மற்றும் வளிமண்டல அறிவியலில் ஆராய்ச்சியை எளிதாக்குவதற்கு தேசிய கண்காணிப்பு வசதிகளை வழங்குவது எதனுடய முக்கிய குறிக்கோள்?
- Generated Answer: வானியல், வானியற்பியல் மற்றும் வளிமண்டல அறிவியல் ஆராய்ச்சியை எளிதாக்குவதற்கு தேசிய கண்காணிப்பு
- Actual Answer: ஆர்யபட்டா வானிலைக் கண்காணிப்பு அறிவியல்கள் ஆராய்ச்சி நிறுவனம்

In both examples, the model repeats part of the question as its answer, highlighting its limitations in providing precise, contextually accurate answers in the absence of task-specific training or guidance.

4.1.1.2 Few-Shot Prompting

Few-shot prompting enhances the model's performance by providing it with a set of task-specific examples (context) along with the new question. These examples serve as a mini training

set embedded within the prompt, helping the model understand the expected format and context of the task.

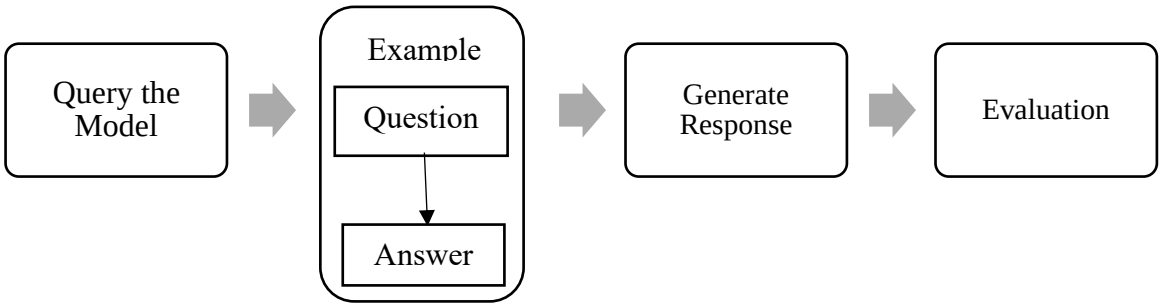

Fig4. Few-Shot Prompting

Example (Malayalam):

Context:

- Example 1:
    - Question: ഹാരപ്പൻ കോട്ടയുടെ തെക്ക് ഭാഗത്ത് കണ്ടെത്തിയ പെയിന്റിംഗുകൾ എന്തെല്ലാമാണ് സൂചിപ്പിക്കുന്നത്?
    - Answer: ഹാരപ്പൻ കോട്ടയുടെ തെക്ക് ഭാഗത്ത് കണ്ടെത്തിയ പെയിന്റിംഗുകൾ, ഹരപ്പൻ ജീവിതരീതിയിലേക്കുള്ള ഭക്ഷണത്തിന്റെ ശീലങ്ങളെ സൂചിപ്പിക്കുന്നു.
- Example 2:
    - Question: ഹാരപ്പൻ സംസ്കാരം ഏത് ഗ്രന്ഥത്തിലെ തത്വങ്ങൾ ആണ് ഉൾക്കൊള്ളുന്നത്?
    - Answer: ഹാരപ്പൻ സംസ്കാരത്തെ ബന്ധപ്പെട്ട ഗ്രന്ഥങ്ങൾ, സംസ്കൃതിയുടെ അടിസ്ഥാന സങ്കല്പങ്ങൾ ഉൾക്കൊള്ളുന്നു.

New Question:

- Question: ഹാരപ്പൻ കോട്ടയുടെ തെക്ക് ഭാഗത്ത് കണ്ടെത്തിയ പെയിന്റിംഗുകൾ, ഇത് ഹരപ്പൻ ജീവിതരീതിയിലേക്കുള്ള ആരുടെ വരവിനെ സൂചിപ്പിക്കുന്നു?
- Generated Answer: ഹാരപ്പൻ കോട്ടയുടെ തെക്ക് ഭാഗത്ത് കണ്ടെത്തിയ പെയിന്റിംഗുകൾ, ഹരപ്പൻ സംസ്കാരത്തിലെ വൈവിധ്യമാർന്ന ഭക്ഷണ ശീലങ്ങളെയും സൂചിപ്പിക്കുന്നു.
- Actual Answer: പരദേശിജീവിതരീതിയുടെ.

Example (Tamil):

Context:

- Example 1:
    - Question: இந்தியாவில் ரோபோடிக்ஸ் மற்றும் இயந்திரக் கற்றலுக்கான முக்கிய ஆராய்ச்சி மையம் எது?
    - Answer: சென்னை தொழில்நுட்ப நிறுவனம் (IIT சென்னை)
- Example 2:
    - Question: இந்திய விண்வெளி ஆய்வுகள் எந்த அமைப்பின் கீழ் மேற்கொள்ளப்படுகின்றன?
    - Answer: இந்திய விண்வெளி ஆய்வு நிறுவனம் (ISRO)

New Question:

- Question: வானியல், வானியற்பியல் மற்றும் வளிமண்டல அறிவியலில் ஆராய்ச்சியை எளிதாக்குவதற்கு தேசிய கண்காணிப்பு வசதிகளை வழங்குவது எதனுடய முக்கிய குறிக்கோள்?
- Generated Answer: அறிவியலில் தேசியப் பேட்டு வானியல் என்ற வானியற்பியல் மற்றும் வளிமண்டலமான கண்காணிப்பு வழங்குவ
- Actual Answer: ஆர்யபட்டா வானிலைக் கண்காணிப்பு அறிவியல்கள் ஆராய்ச்சி நிறுவனம்

Here, the model's capacity to produce contextually relevant responses is much enhanced by the addition of instances. It demonstrates the promise of few-shot prompting for low-resource languages by demonstrating how it greatly increases the contextual accuracy of the responses. Few-shot prompting is an effective method for low-resource languages because it fills the gap between zero-shot prompting and fine-tuning.

4.1.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a hybrid system that unifies the capabilities of language models with external retrieval machinery to enrich the quality of their responses. Even though language models such as Gemma 2B, Open Llama 3.1B, and T5 Base prove excellent in generating con-textually relevant answers, they may lack specific details, particularly in low-resource or domain-specific settings. RAG addresses the problem by allowing the models to retrieve extra knowledge from an external dataset, hence further enhancing their ability to create precise and contextually rich answers.

In this experiment, RAG is used to improve the question-answering performance of Malayalam and Tamil by combining the language models with retriever modules that search through external datasets, namely mc4-ml (Malayalam) and mc4-ta (Tamil). This is to ensure that the models have access to a more extensive knowledge base, which is helpful in handling complex or underrepresented queries in Dravidian languages.

Workflow of RAG System

The RAG methodology can be understood through the following sequential workflow and from fig5:

1. Query Processing: It begins with a query—such as a question or prompt requiring a response to it
2. Embedding Model: The query is passed through an embedding model (**paraphrase-multilingual-MiniLM-L12-v2**) to create a vector representation, capturing its semantic meaning.
3. Vector Database Retrieval: Using **ChromaDB**, the query vector searches for semantically similar contexts stored as vectors in the database. The top matches are retrieved. ChromaDB provides an efficient mechanism to retrieve semantic contexts from the multilingual corpora.
4. Retrieved Contexts: These contexts are rich in domain-specific information and are sent to the generator.
5. LLM Response Generation: The generator processes the query and the retrieved contexts to produce a response, coupling internal knowledge with the external data.
6. Final Response: The output is an informed and accurate response that integrates pre-trained model knowledge with retrieved domain-specific information.

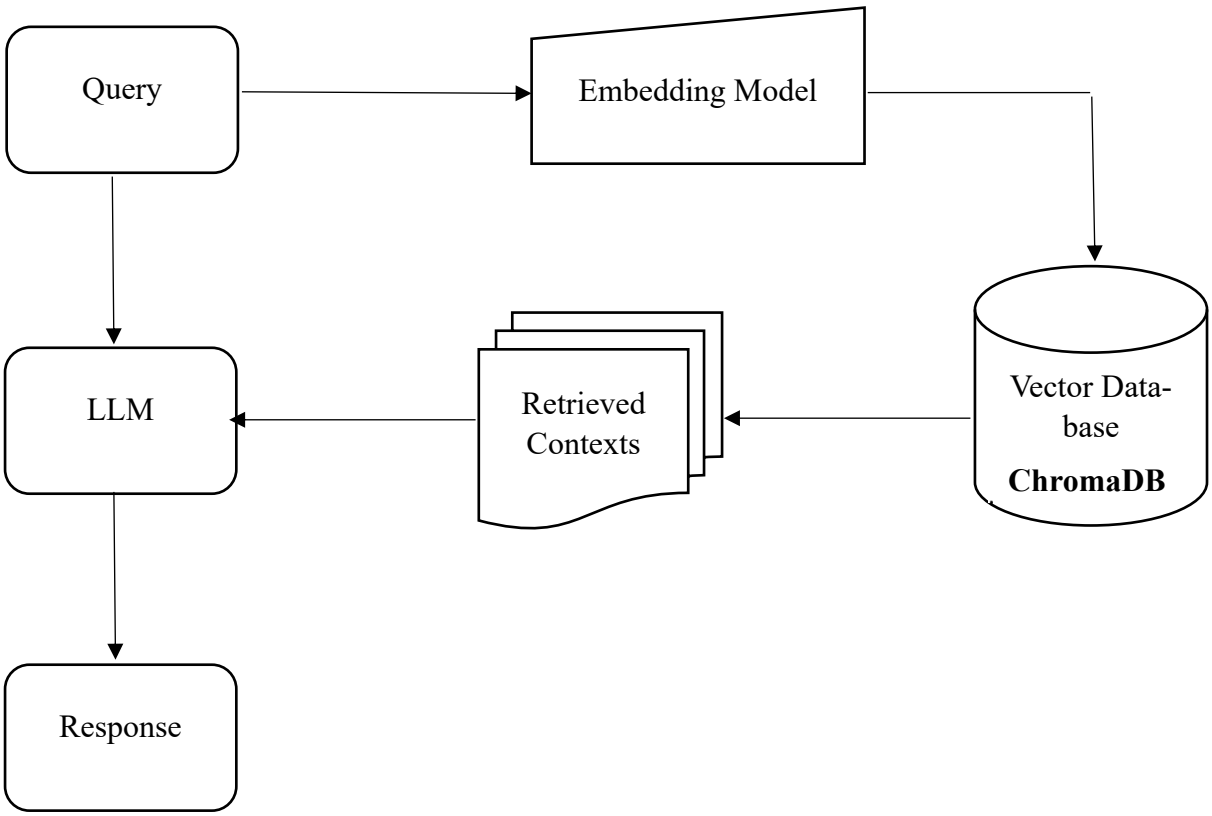

Fig5. Workflow of RAG

The Retrieval-Augmented Generation (RAG) approach involves two critical components—Retriever and Generator that work together to improve the model's ability to generate accurate, contextual responses. Below is a detailed explanation of each component:

4.1.2.1 Retriever

The Retriever is the knowledge-seeking module of the RAG framework. It primarily fetches relevant information from an external dataset that supplements the knowledge inside the model. In this study, mc4-ml for Malayalam and mc4-ta for Tamil are used as the external datasets. These datasets are large multilingual corpora curated for natural language understanding tasks.

When a question is asked, the retriever scans the external dataset and identifies the most relevant documents or passages based on semantic similarity to the query. For example, if the question is about Harappan culture, then the retriever extracts the passages related to "Harappan," "paintings," and their influence on lifestyle.

Example (Malayalam):

- Query:
  ഹാരപ്പൻ കോട്ടയുടെ തെക്ക് ഭാഗത്ത് കണ്ടെത്തിയ പെയിന്റിംഗുകൾ, ഇത് ഹരപ്പൻ ജീവിതരീതിയിലേക്കുള്ള ആരുടെ വരവിനെ സൂചിപ്പിക്കുന്നു?
- Retrieved Context from mc4-ml:

  "ഹാരപ്പൻ കോട്ടയുടെ തെക്ക് ഭാഗത്ത് കണ്ടെത്തിയ ചില പെയിന്റിംഗുകൾ, അവർ പരദേശികളുമായി ഉണ്ടായിരുന്ന ബന്ധങ്ങളും അവരുടെ ജീവിതരീതിയിലേക്ക് കണ്ടുകെട്ടലുകളും പ്രതിപാദിക്കുന്നു."

Example (Tamil):

- Query:
  வானியல், வானியற்பியல் மற்றும் வளிமண்டல அறிவியலில் ஆராய்ச்சியை எளிதாக்குவதற்கு தேசிய கண்காணிப்பு வசதிகளை வழங்குவது எதனுடைய முக்கிய குறிக்கோள்?
- Retrieved Context from mc4-ta:
  "ஆர்யபட்டா வானியற்பியல் ஆராய்ச்சி நிறுவனம் வானியல் மற்றும் வளிமண்டல அறிவியலுக்கான ஆராய்ச்சியை மேம்படுத்துவதற்கான முக்கிய தேசிய வசதிகளை வழங்குகிறது."

The retriever makes sure the retrieved context closely aligns with the user's query, thereby reducing ambiguity and making the final response more relevant. This is particularly crucial for low-resource languages such as Malayalam and Tamil, wherein a particular dataset might not provide extensive coverage of diverse topics.

4.1.2.2 Generator

The generator is the central language model responsible for generating the final response. Once the retriever has fetched the relevant documents or context, the generator combines this re-tried information with the input query to form a coherent and accurate answer.

The generator operates by augmenting its internal knowledge (pre-trained during large-scale language modeling) with the retrieved external data. This dual reliance ensures that the generated response is both informed by the model’s linguistic understanding and enriched with additional domain-specific details.

Example (Malayalam):

- Query:
  ഹാരപ്പൻ കോട്ടയുടെ തെക്ക് ഭാഗത്ത് കണ്ടെത്തിയ പെയിന്റിംഗുകൾ, ഇത് ഹരപ്പൻ ജീവിതരീതിയിലേക്കുള്ള ആരുടെ വരവിനെ സൂചിപ്പിക്കുന്നു?

- Generated response:

  "ഹാരപ്പൻ കോട്ടയുടെ തെക്ക് ഭാഗത്ത് കണ്ടെത്തിയ പെയിന്റിംഗുകൾ, ഹരപ്പൻ ജീവിതരീതിയിലേക്ക് പരദേശികളുടെ ശീലങ്ങളെയും സംസ്കാരങ്ങളെയും സൂചിപ്പിക്കുന്നു."

- Actual Answer:
  "പരദേശിജീവിതരീതിയുടെ."

Example (Tamil):

- Query:
  வானியல், வானியற்பியல் மற்றும் வளிமண்டல அறிவியலில் ஆராய்ச்சியை எளிதாக்குவதற்கு தேசிய கண்காணிப்பு வசதிகளை வழங்குவது எதனுடைய முக்கிய குறிக்கோள்?

- Generated Response:

  "ஆர்யபட்டா வானியற்பியல் ஆராய்ச்சி நிறுவனம் வானியல், வானியற்பியல் மற்றும் வளிமண்டல அறிவியலுக்கான தேசிய வசதிகளை வழங்குவதன் மூலம் ஆராய்ச்சியை மேம்படுத்துகிறது."

- Actual Answer:

  "ஆர்யபட்டா வானிலைக் கண்காணிப்பு அறிவியல்கள் ஆராய்ச்சி நிறுவனம்."

The unifying and integrating framework of RAG thus improves the ability of a system to tackle linguistic diversities and resource restrictions due to languages such as Malayalam and Tamil. This is because the retriever retrieves domain-specific information while the generator integrates it aptly, therefore leading to the enrichment of this knowledge with accurate responses. This approach surpasses zero-shot and few-shot prompting by including external knowledge, which allows the models to address linguistic complexities and knowledge gaps inherent to low-resource languages. The examples above highlight the potential of RAG in significantly improving QA systems for Dravidian languages.

4.1.3 Low Rank Adaptation (LoRA)

Low-Rank Adaptation (LoRA) is a technique for fine-tuning large language models (LLMs) that has gained significant attention for its efficiency and effectiveness, especially in re-source-constrained settings. Unlike traditional fine-tuning, which updates all model parameters, LoRA freezes the original model weights and introduces task-specific trainable parameters in the form of low-rank updates. This approach drastically reduces computation-al costs while maintaining high performance, making it particularly suitable for adapting LLMs to low-resource languages like Malayalam and Tamil.

Large language models such as Gemma 2B, Open Llama 3.1B, and T5 Base are pre-trained on large corpora in high-resource languages. These models struggle with low-resource languages mainly because of data unavailability and computational limitations in fine-tuning the whole model for specific domain or language-related tasks. Fine-tuning a whole model for specific domains or language-specific tasks requires huge hardware and memory.

LoRA overcomes these challenges by proposing task-specific adaptations without modifying the core weights of the pre-trained model. This is highly advantageous for QA systems in Malayalam and Tamil because the datasets are small and computational resources are limited. In this way, QA models can adapt to these languages effectively, yet maintain scalability and efficiency.

4.1.3.1 Working of LoRA

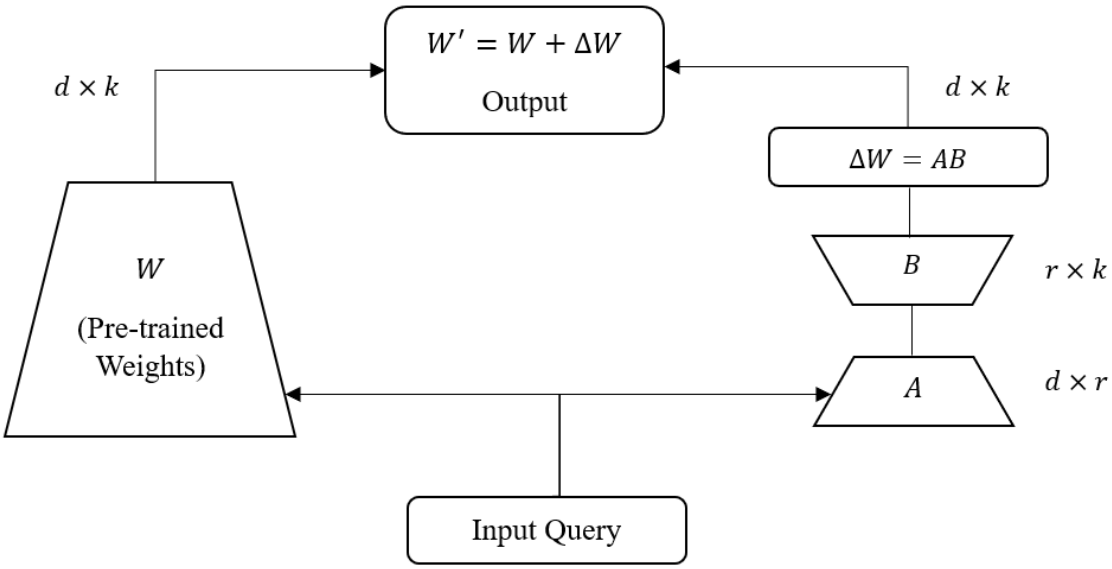

Fig6. Working of LoRA

LoRA is based on the low-rank decomposition of weight matrices. For Transformer-based language models, the computationally expensive parts include the query, key, and value projection matrices that are used in the self-attention mechanism, denoted as W_Q, W_K, $W_V \in R^{d\times k}$, where d is the input dimension, and k is the output dimension. The projected representations are computed as:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

where $X \in R^{n\times d}$ is the input to the layer.

LoRA modifies the weight matrices W_Q, W_K, W_V by adding a low-rank update, defined as:

$$W' = W + \Delta W, \qquad \Delta W = AB$$

where:

- $A \in R^{d\times r}$ and $B \in R^{r\times k}$ are trainable low-rank matrices.
- $r \ll \min(d,k)$, ensuring computational efficiency.

For example, if $W_Q \in R^{1024\times512}$ and r=8, the number of trainable parameters for ΔW is:

$$r \cdot (d + k) = 8 \cdot (1024 + 512) = 12{,}288$$

Instead, the traditional fine-tuning updates d · k=524,288 parameters. The mathematical working of LoRA is presented in fig6. Due to the reduction of parameters, LoRA is highly efficient. With LoRA, one can perform fine-tuning on consumer-grade hardware without losing model performance. During training, the gradients are computed only for the low-rank matrices A and B:

$$A \leftarrow A - \alpha \frac{\partial L}{\partial A}, \qquad B \leftarrow B - \alpha \frac{\partial L}{\partial B}$$

where L is the loss function, and α is the learning rate. This ensures that the model efficiently learns task-specific knowledge without modifying the frozen pre-trained weights.

LoRA is a paradigm shift in fine-tuning large language models, with computationally efficient and scalable adaptation of QA systems to low-resource languages. LoRA reduces the resource requirements of the update while improving the adaptability of pre-trained models to new tasks and domains. It has been applied to QA for Malayalam and Tamil, where it shows promise in closing the linguistic resource gap for more inclusive and effective language technologies.

4.1.4 Quantized Low-Rank Adaptation (QLoRA)

QLoRA is an advanced extension of LoRA, which exploits parameter quantization to further reduce memory usage and computational overhead. Combining the low-rank adaptation with quantization, QLoRA allows efficient fine-tuning of large-scale language models for specific tasks, such as QA in low-resource languages like Malayalam and Tamil.

The key innovation in QLoRA is the compression of the frozen weights of the model into a lower-bit representation, such as 4-bit or 8-bit integers, while introducing lightweight low-rank adapters. This dual optimization reduces resource requirements by a large amount while maintaining high accuracy, making it particularly suitable for resource-constrained environments.

Quantized Low-Rank Adaptation (QLoRA) is a revolutionary fine-tuning method for large language models, especially in resource-constrained environments. Its main strength is that it can significantly reduce the memory and computational requirements of traditional fine-tuning methods. QLoRA compresses the frozen weights of a pre-trained model into lower-bit representations, such as 4-bit integers, thereby reducing the storage demand. This is of particular use for deploying and training large models on machines with relatively limited hardware capacities, like GPUs with lower memory or edge devices.

Adding trainable low-rank matrices ensures the adaptability of QLoRA with no loss of integrity from the quantized weights. That way, it could allow QLoRA to benefit from such extensive pretraining and then properly fine-tune the model for specific tasks like QA in resource-poor languages such as Malayalam and Tamil. Moreover, the only updates of the low-rank parameters result in faster training times and lower computational overheads than that of finetuning all the parameters of the model. These benefits make it a highly efficient and scalable solution,

allowing researchers and developers to address the linguistic complexities and data scarcity associated with low-resource languages.

Another key strength of QLoRA is its bridging knowledge gaps. Enabling large models to retain pre-trained knowledge while incorporating task-specific adaptations, QLoRA improves the model's ability to produce contextually rich and accurate responses. This is especially important for QA systems in languages where there is a scarcity of annotated datasets; the ability to extrapolate from pre-trained knowledge becomes the most important feature. In essence, QLoRA democratizes the application of advanced language models in settings with con-strained resources, and thus opens new avenues for research and development in multilingual and low-resource scenarios.

4.1.4.1 Working of QLoRA

The QLoRA process starts with the quantization of a pre-trained model's weights that can be easily understood from fig7. The original weights usually stored as 32-bit floating-point numbers are compressed in low-bit formats, such as 4-bit integers (NF4) and 8-bit quantization. NF4 optimizes the representation of the weight distributions, making sure that there is a minimal loss of precision despite the drastic reduction in memory footprint. The quantization process mathematically is:

$$Q(W) = Quantize(W, NF4)$$

where $Q(W)$ represents the quantized weights.

This quantization step severely reduces the memory footprint of frozen weights, al-lowing to work on large models on hardware that has limited capacity. In spite of the precision degradation, quantization ensures to the fullest extend possible that the original information contained within the weights is preserved in the model, hence ensuring that the pre-trained abilities can be maintained.

To adapt the model to new tasks, QLoRA introduces trainable low-rank matrices, A and B, which are designed to capture task-specific information. These matrices are much smaller than the original weight matrices, and their product,

$$\Delta W = AB$$

represents the learned adaptation. During training, only the parameters in A and B are updated, leaving the quantized frozen weights untouched. This approach ensures that the model efficiently learns task-specific adaptations while retaining its pre-trained knowledge base.

At inference, the model combines the quantized weights $Q(W)$ with the learned low-rank adaptations (ΔW) to compute the final weights,

$$W_{final} = \mathrm{Q(W)} + \Delta \mathrm{W}$$

It combines the pre-trained knowledge that the model has learned through prior training and the task-specific refinements introduced during fine-tuning. The system achieves low computational and memory costs for deploying and fine-tuning the large language models, particularly suitable for QA tasks in low-resource lan-guages like Malayalam and Tamil. QloRA has brought an innovative solution towards such challenging but very much needed applications with its quantized base model along with light-weight adaptations.

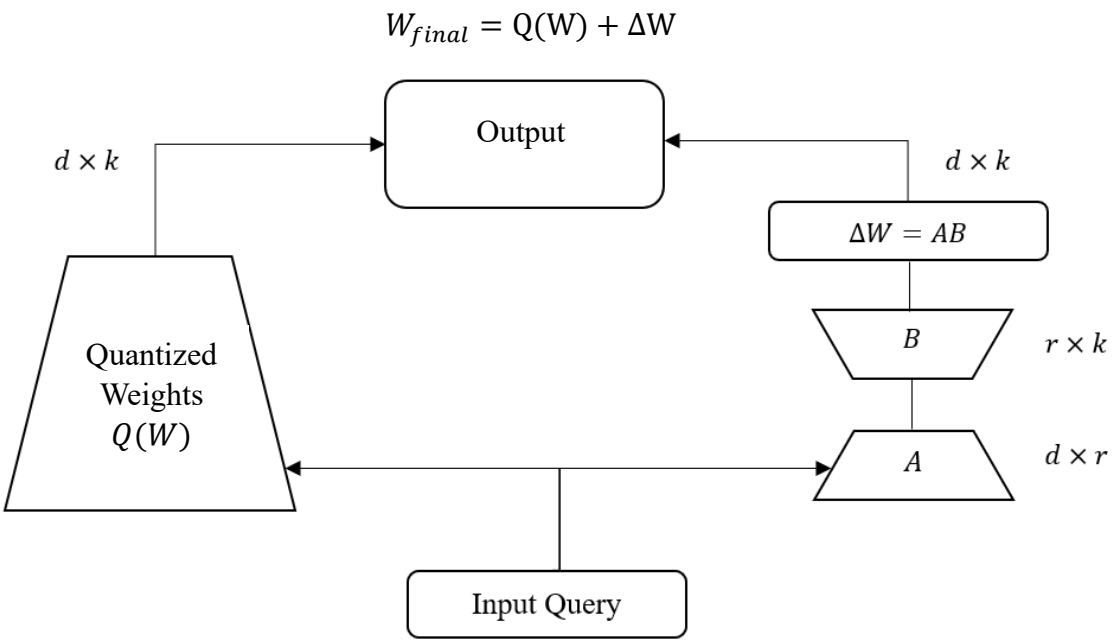

Fig7. Working of QLoRA

5. Experiments and Discussions

5.1 Experimental Setup

Hardware:
The experiments were performed on three NVIDIA A100 GPU, assisted by 21 CPUs, 96GB of RAM and 120GB VRAM.

Software:
The experiments were conducted in Python 3.10 with PyTorch 2.0 and Hugging Face Transformers 4.35. The RAG system used FastText for vector embedding and ChromaDB for maintaining the vector database. Data pre-processing was done using pandas 2.1, numpy 1.25, and nltk 3.9, with LoRA and QLoRA modules integrated for efficient fine-tuning. The quantization was managed using the bitsandbytes library for 8-bit quantization implementation. The whole experiment was executed on Ubuntu 20.04 LTS. Dependencies were managed via pip.

5.2 Results and Discussions

5.2.1 Prompting

In this section, we analyze the performance of zero-shot and few-shot prompting strategies for Tamil and Malayalam question-answering tasks, employing three tiny language models: Open Llama 3.1B, Gemma 2B, and T5 Base. We examine how the performance varies for these models on interpretations and responses to questions by way of the two paradigms and measure their performance outputs by standard metrics: BLEU, ROUGE, Cosine Similarity and F1 Score.

5.2.1.1 Zero-Shot Prompting

Zero-shot prompting involves providing the model with only the query in the form of a natural language question, without any additional examples to guide its response. The prompt structure is simple:

Prompt Design:

For Malayalam:

> Question: 1950 ൽ ഇന്ത്യ വിഭജനത്തിനുശേഷം, ഇന്ഡസ് വാലി പ്രദേശത്ത് വിപുലമായ പഠനങ്ങൾ നടത്തുകയും കൂടുതൽ അവശിഷ്ടങ്ങൾ കണ്ടെത്തുകയും ചെയ്തതാര്?
>
> Generated Answer:

For Tamil:

> Question: வானியல், வானியற்பியல் மற்றும் வளிமண்டல அறிவியலில் ஆராய்ச்சியை எளிதாக்குவதற்கு தேசிய கண்காணிப்பு வசதிகளை வழங்குவது எதனுடைய முக்கிய குறிக்கோள்?
>
> Generated Answer:

Results:

Metric \ Model	Gemma 2B	Open-Llama 3.1B	T5-base
BLEU	0	0	0
ROUGE1	0	0	0
ROUGE2	0	0	0
ROUGEL	0	0	0
F1-score	0	0	0
Cosine Similarity	0	0	0

Table3. Result of zero-shot prompting in Malayalam

Zero-shot setting produced suboptimal performance in all models and for both languages. For Tamil and Malayalam, BLEU, ROUGE, and Cosine Similarity scores were zero, and the generated answers were found to have no meaningful resemblance to the ground truth. Even the cosine similarity values were low or negligible, and no semantic alignment between the generated responses and expected answers was observed.

Metric \ Model	Gemma 2B	Open-Llama 3.1B	T5-base
BLEU	0	0	0
ROUGE1	0	0	0
ROUGE2	0	0	0
ROUGEL	0	0	0
F1-score	0	0	0
Cosine Similarity	0	0	0

Table4. Result of zero-shot prompting in Tamil

This result shows how hard it is to harness small language models for a zero-shot setup, in terms of returning correct and contextually relevant answers in low-resource languages, mainly because examples are lacking. Responses will be either vague or totally wrong due to the lack of direction by task-specific information, whereas the model depends only on its pre-trained knowledge.

5.2.1.2 Few-Shot Prompting

Few-shot prompting refers to giving the model one or more examples of question-answer pairs before a new question is presented to the model. This is the context and guide where the model can make inferences regarding the task structure to come up with more meaningful answers.

Prompt Design:

For Malayalam:

Example 1:

Question: ഹാരപ്പൻ കോട്ടയുടെ തെക്ക് ഭാഗത്ത് കണ്ടെത്തിയ പെയിന്റിംഗുകൾ എന്തെല്ലാമാണ് സൂചിപ്പിക്കുന്നത്?

Answer: ഹാരപ്പൻ കോട്ടയുടെ തെക്ക് ഭാഗത്ത് കണ്ടെത്തിയ പെയിന്റിംഗുകൾ, ഹരപ്പൻ ജീവിതരീതിയിലേക്കുള്ള ഭക്ഷണത്തിന്റെ ശീലങ്ങളെ സൂചിപ്പിക്കുന്നു.

Example 2:

Question: ഹാരപ്പൻ സംസ്കാരം ഏത് ഗ്രന്ഥത്തിലെ തത്വങ്ങൾ ആണ് ഉൾക്കൊള്ളുന്നത്?

Answer: ഹാരപ്പൻ സംസ്കാരത്തെ ബന്ധപ്പെട്ട ഗ്രന്ഥങ്ങൾ, സംസ്കൃതിയുടെ അടിസ്ഥാന സങ്കല്പങ്ങൾ ഉൾക്കൊള്ളുന്നു.

New Question:

Question: ഹാരപ്പൻ കോട്ടയുടെ തെക്ക് ഭാഗത്ത് കണ്ടെത്തിയ പെയിന്റിംഗുകൾ, ഇത് ഹരപ്പൻ ജീവിതരീതിയിലേക്കുള്ള ആരുടെ വരവിനെ സൂചിപ്പിക്കുന്നു?

Generated Answer: ഹാരപ്പൻ കോട്ടയുടെ തെക്ക് ഭാഗത്ത് കണ്ടെത്തിയ പെയിന്റിംഗുകൾ, ഹരപ്പൻ ജീവിതരീതിയിലേക്കുള്ള പല ബാഹ്യ സ്വാധീനങ്ങളുടെ വരവിനെ സൂചിപ്പിക്കുന്നു.

This response shares the pattern of answers in examples, focusing on insights in Harappan life in paintings while incorporating a logical understanding of external influences, just as the new question seems to imply.

For Tamil:

Example 1:

Question: இந்தியாவில் ரோபோடிக்ஸ் மற்றும் இயந்திரக் கற்றலுக்கான முக்கிய ஆராய்ச்சி மையம் எது?

Answer: சென்னை தொழில்நுட்ப நிறுவனம் (IIT சென்னை)

Example 2:

Question: இந்திய விண்வெளி ஆய்வுகள் எந்த அமைப்பின் கீழ் மேற்கொள்ளப்படுகின்றன?

Answer: இந்திய விண்வெளி ஆய்வு நிறுவனம் (ISRO)

New Question:

Question: வானியல், வானியற்பியல் மற்றும் வளிமண்டல அறிவியலில் ஆராய்ச்சியை எளிதாக்குவதற்கு தேசிய கண்காணிப்பு வசதிகளை வழங்குவது எதனுடைய முக்கிய குறிக்கோள்?

Generated Answer: இந்தியாவின் வானியல் ஆய்வு கழகம் (IIA).

This answer follows the logical pattern from the examples and provides concise answers where key institutions related to the question domain. In this experiment, two-shot prompting (with two examples) was used.

Results:

Metric \ Model	Gemma 2B	Open-Llama 3.1B	T5-base
BLEU	1.53×10^{-231}	9.59×10^{-232}	0
ROUGE1	0	0	0
ROUGE2	0	0	0
ROUGEL	0	0	0
F1-score	0.5	0.133	0
Cosine Similarity	1.0	0.323	0

Table5. Results of few-shot prompting in Malayalam

Metric \ Model	Gemma 2B	Open-Llama 3.1B	T5-base
BLEU	1.00×10^{-231}	1.02×10^{-231}	0
ROUGE1	0	0	0
ROUGE2	0	0	0
ROUGEL	0	0	0
F1-score	0.118	0.125	0
Cosine Similarity	0.401	0.352	0

Table6. Results of few-shot prompting in Tamil

The few-shot prompting evaluation gave a good idea about how Open Llama, Gemma, and T5 Base perform in such conditions, as they are provided with just a few contextual examples. For the Tamil dataset, Open Llama reported a Cosine Similarity of 0.352 and an F1 score of 0.125. These scores are slightly higher than the zero-shot ones, but still, quite low, indicating a meagre improvement even after having examples. Gemma, however, obtained a Cosine Similarity of 0.401, outperforming Open Llama in semantic relevance but lagging in F1 score (0.118). This performance gap reflects the impact of the pretraining corpus and the model's alignment with Tamil's linguistic patterns. The T5 Base model did not produce meaningful responses since its BLEU and Cosine Similarity scores were 0 across all metrics for Tamil.

On the Malayalam dataset, Open Llama scored 0.323 in terms of Cosine Similarity along with an F1 score of 0.133. Still limited, this was just a shade better than that on the Tamil dataset, implying a relatively better fit for Malayalam linguistic structures. The best-performing model on Malayalam was Gemma, scoring a Cosine Similarity of 1.000 with an F1 score of 0.5, way ahead of other models. This means that Gemma's pre-training corpus may have contained more Malayalam data or that the syntactic simplicity of the language compared to Tamil contributed to the gain. T5 Base was weak throughout both datasets, indicating that it is not very effective in adapting to low-resource Dravidian languages unless further finetuned.

The poor performance of the two models, Open Llama and T5 Base can mainly be attributed to lack of sufficient representation of Dravidian languages in their respective pretraining corpus, in which case, the somewhat relatively better performance of Gemma in Malayalam speaks more of its possible reception to a more representative dataset or linguistic alignment. Tamil and Malayalam are really challenging for large language models because they have rich morphology, agglutinative nature, and free word or der, that makes it rather difficult to generate lexically and semantically accurate answers, particularly in zero-shot settings. The few-shot setting provides much more context; however, the quality and the number of examples given drastically limit how much one might improve it, particularly if the resource is scarce: the most important feature being diverse, high-quality prompts. Further, the generalized architectures of Open Llama and T5 Base fail to capture the linguistic nuances of Dravidian languages, whereas the performance of Gemma indicates the need for domain-specific adaptations and pretraining strategies to improve the capabilities of the model.

5.2.2 Retrieval Augmented Generation (RAG)

For the Malayalam dataset, in Table7 we have observed a lot of difference between the performances of Open Llama, Gemma, and T5 Base. All three models showed ROUGE scores of zero for both ROUGE-1 and ROUGE-L, meaning that the models failed to generate substantial overlap with the ground truth answers in terms of recall, precision, or F1 score. This poor performance may be due to the inability of these models to handle the morphological and syntactic complexities inherent in the Malayalam language.

However, there were differences of performance in other metrics. While Open Llama scored with a cosine similarity of 0.5676, Gemma scored slightly better at 0.58. All these values indicate that it was not possible for either of the models to capture the reference answers' precise text but could generate at least some contextual similarity. The relatively better performance of Gemma could be due to its closer alignment with the Malayalam language, especially because the pretraining was done with more suitable data or techniques in handling low-resource languages.

T5 Base scored a cosine similarity of 0 and zero performance in all other metrics, such as ROUGE and F1. This result further supports the conclusion that T5 Base, a general-purpose model, struggles significantly with Malayalam, likely due to its generic pretraining dataset that may not effectively cover the intricacies of the Dravidian language.

BERTScore is an evaluation metric that employs pre-trained BERT embeddings to measure the semantic similarity between a generated text and a reference text, giving a more context-aware assessment than traditional metrics like BLEU and ROUGE. BERTScore[26] was better for all models, having values around 0.86 to 0.87, meaning that the models could still generate semantically similar responses to the reference answers, though they did not match precisely.

Model / Metric	Gemma 2B	Open-Llama 3.1B	T5-base
BLEU	0.7	0.12	0
ROUGE1	0	0	0
ROUGEL	0	0	0
F1-score	0.15	0	0
Cosine Similarity	0.58	0.5676	0
BERT_Score Precision	0.85	0.8649	0.84
BERT_Score Recall	0.9	0.8949	0.85
BERT_Score F1-score	0.8797	0.8797	0.86

Table7. Results of RAG for Malayalam

The performance for the Tamil dataset was also similar. Gemma performed better than T5 Base and Open Llama with a cosine similarity score of 0.58, much better than T5 Base 0 and Open Llama (0.5676). Again, like Malayalam, none of the models could get an exact match to the

ground truth answers, as dealing with Tamil's agglutinative structure and free word order is tough. These linguistic features make it difficult for the models to align perfectly with the reference answers, even when their semantic similarity is relatively high.

The performance of Gemma in Tamil also shows the effect of pretraining on low-resource languages. Its better performance on BERTScore (0.85 to 0.89) shows that it was able to generate more contextually relevant responses, even though it was not exactly similar to the reference answers..

T5 Base and Open Llama yielded results that were similar; Precision BERTScores scored 0.92 while the Recall BERTScores 0.95, which would demonstrate their ability to capture semantic essence of the question as well as context. However, none of the models did the best in generating the very accurate responses since it indicated by their low F1 Scores and lack of ROUGE score. These models probably performed very well on understanding the intent of the question but did not reproduce the precise phrasing or structure needed to get high-quality answers.

Model / Metric	Gemma 2B	Open-Llama 3.1B	T5-base
BLEU	0	0	0
ROUGE1	0	0	0
ROUGEL	0	0	0
F1-score	0.0015	0.0013	0.0028
Cosine Similarity	0.587	0.562	0
BERT_Score Precision	0.8648	0.9258	0.9258
BERT_Score Recall	0.9151	0.9527	0.9502
BERT_Score F1-score	0.8892	0.9390	0.9378

Table8. Results of RAG for Tamil

Several design decisions in the context retrieval and embedding process of the RAG pipeline may be responsible for the below observed performance. ChromaDB was used to store and retrieve embeddings generated by the fastText model for Malayalam. However, quality and the representation of the embeddings are probably not good enough to take care of the morphological richness and structural variation of the Malayalam language. While ChromaDB and fastText enable one to store and retrieve large datasets in embeddings, the quality of such embeddings determines retrieval effectiveness. Moreover, the SentenceTransformer and paraphrase-multilingual-MiniLM-L12-v2 model, used to manage multilingual contexts, would have likely affected the low success rates in giving accurate answers, especially when the language resources are low like in the case of Malaya-lam and Tamil.

In addition, the retrieval process, which was based on ChromaDB for obtaining contexts, may not necessarily have retrieved the most accurate contexts for each query that was made. This inadequacy in context retrieval explains why the models, producing responses with some

degree of semantic similarity, could hardly produce highly accurate answers. That is, the retrieved contexts lack the linguistic cues necessary for the proper generation of the correct answers.

The high BERTScore in the results was mainly due to the model's ability to capture semantic similarity between the generated responses and reference answers rather than focusing on the word match. Unlike other metrics that measure exact matches, such as ROUGE or Exact Match, BERTScore is based on contextual embeddings from models like BERT, which are trained to understand words and phrases in their specific con-texts. This enables BERTScore to evaluate the generated response in relation to how well it represents the meaning, even if not identical, in comparison to the reference.

In the case of low-resource languages like Malayalam and Tamil, where an exact word match might be difficult because of their complex morphology and structure, BERTScore still manages to give a high value because it focuses on the semantic content of the response rather than its exact wording. For example, if the ground truth answer in Tamil for a question is "அவன் சாப்பாடு சாப்பிட்டார்," (He ate food), a model might come up with "அவர் உணவு பருகினார்" (He consumed food). Although the words are different, models can still understand that both sentences have the same semantic meaning. BERTScore will, therefore, score very high because it picks on the general meaning intended to be conveyed by the words even though their forms differ.

Similarly, in the Malayalam dataset, suppose the reference answer for a question is:

Question: "സൂര്യൻ എങ്ങനെ ഊർജ്ജം ഉൽപാദിപ്പിക്കുന്നു?" (How does the sun produce energy?)

Reference Answer: "സൂര്യൻ ഊർജ്ജം സൃഷ്ടിക്കുന്നതിനായി വിന്യാസം പരിവർത്തനവും പ്രകാശം പ്രകിയാബന്ധുവും ഉപയോഗിക്കുന്നു." (The sun uses nuclear fusion and light energy conversion to produce energy.)

The model might generate the following response:

Generated Answer: "സൂര്യൻ ഊർജ്ജം സൃഷ്ടിക്കാൻ വിന്യാസ പരിവർത്തനവും പ്രകാശം ഉപയോഗിക്കുന്നു." (The sun uses fusion and light to produce energy.)

Even though the exact words differ (e.g., "നൂക്ലിയർ ഫ്യൂഷൻ" vs "വിന്യാസം" or "light energy conversion" vs "പ്രകാശം"), both answers have similar context. The BERTScore would be high in this case because it compares the semantic meaning of both sentences, capturing the essence of energy production by the sun, even if the phrasing differs.

In our experiment, also the context retrieval process applied in the RAG pipeline contributes to the high BERTScore. The process retrieves 6 relevant contexts from external resources such as Wikipedia or CC100 using ChromaDB, which are then fed into the model as part of the prompt. For instance, if the question asked is "Why do trees require sunlight?," then the retrieved contexts might include passages explaining the role of photosynthesis, the use of sunlight in plant growth, and so on. The contexts help provide the model with more information that might aid it in generating a more contextually relevant response. Although the actual text of the response could be different from the reference, the model can still make use of the context to give an answer that is semantically aligned with the ground truth. The high BERTScore

indicates that the model has effectively captured this semantic similarity, even though it didn't match the phrasing exactly. It is the BERTScore that mainly captures the high-level semantic alignment of the generated answers with the reference answers. This indicates that although language-specific challenges might cause slight variations in wording, still the models were able to produce responses that matched in meaning with the reference answers closely, thus yielding high BERTScores.

5.2.3 Low Rank Adaptation (LoRA)

Model / Metric	Gemma 2B	Open-Llama 3.1B	T5-base
BLEU	0.65	0.1	0.02
ROUGE1	0.02	0.01	0
ROUGEL	0.01	0.01	0
Precision	0.83	0.85	0.16
Recall	0..87	0.88	0.77
F1-score	0.85	0.865	0.39
Cosine Similarity	0.56	0.99	0

Table9. Results of LoRA for Malayalam

Model / Metric	Gemma 2B	Open-Llama 3.1B	T5-base
BLEU	0.4	0.01	0.02
ROUGE1	0.01	0.03	0
ROUGEL	0.01	0.02	0
Precision	0.71	0.70	0.53
Recall	0..86	0.95	0.59
F1-score	0.73	0.81	0.64
Cosine Similarity	0.56	0.76	0.28

Table10. Results of LoRA for Tamil

In our experiment, also the process applied during context retrieval in the RAG pipeline contributes to this high BERTScore: 6 relevant contexts retrieved by ChromaDB from external resources, like Wikipedia or CC100, are fed into the model as part of the prompt. For example, if the question asked is "Why do trees need sunlight?, then the retrieved contexts would include passages explaining the role of photosynthesis, how sunlight is used in the growth of plants, and so on. The contexts help provide the model with more information that might help it generate a more contextually relevant response. Although the actual response text may differ from that of the reference, the model still uses the context to generate an answer that is semantically aligned with the ground truth. The high BERTScore indicates that this semantic similarity has been picked up by the model very well, even though the phrasing was not matched exactly. It

is the BERTScore that mainly captures the high-level semantic alignment of the generated answers with the reference answers. This indicates that although language-specific challenges might cause slight variations in wording, still the models were able to produce responses that matched in meaning with the reference answers closely, thus yielding high BERTScores.

In contrast, precision, recall, and F1 scores show that the models are all very strong in semantic alignment. Again, Open-Llama 3.1B achieves a high F1-score score of 0.865, followed closely by Gemma 2B on 0.85; T5-base performs slightly lower at 0.39. These scores show good alignment of the generated answers' meaning with the reference's, even when the wording is very different. The second one is cosine similarity. Open-Llama 3.1B reached an almost perfect score of 0.99 and significantly outperformed other models in contextual relevance, which is a strong indicator of the importance of embedding-based evaluations for QA tasks in low-resource languages.

For instance, in a Malayalam QA task, when queried, "ഹാരപ്പൻ കോട്ടയുടെ തെക്ക് ഭാഗത്ത് കണ്ടെത്തിയ പെയിന്റിംഗുകൾ, ഇത് ഹരപ്പൻ ജീവിതരീതിയിലേക്കുള്ള ആരുടെ വരവിനെ സൂചിപ്പിക്കുന്നു?" (The paintings found in the southern part of the Harappan fort indicate the arrival of whom in Harappan lifestyles?), the actual answer is "പരദേശിജീവിതരീതിയുടെ" (Foreign lifestyles). The LoRA-adapted model generates the response, "അവർ ജീവിതത്തിൽ വിദേശ സ്വാധീനത്തെ പ്രതിപാദിക്കുന്നു" (They indicate foreign influence in life). Although the wording of the generated answer is different, it captures the semantic intent of the actual answer very well, as shown by the high precision and recall scores.

Similarly, in a Tamil example, the question "வானியல், வானியற்பியல் மற்றும் வளிமண்டல அறிவியலில் ஆராய்ச்சியை எளிதாக்குவதற்கு தேசிய கண்காணிப்பு வசதிகளை வழங்குவது எதனுடய முக்கிய குறிக்கோள்?" (What is the primary goal of providing national monitoring facilities for research in astronomy, astrophysics, and atmospheric sciences?) has the actual answer "ஆர்யபட்டா வானிலைக் கண்காணிப்பு அறிவியல்கள் ஆராய்ச்சி நிறுவனம்" (Aryabhata Atmospheric Research Institute). The LoRA-adapted model creattes the response "ஆர்யபட்டா ஆராய்ச்சி நிறுவனம் வானியல் மற்றும் வானியற்பியல் ஆராய்ச்சிக்கான வாய்ப்புகளை அதிகரிக்கிறது" (Aryabhata Research Institute enhances opportunities for research in astronomy and astrophysics). Though the generated answering extends a little bit, the semantic essence of the original is still carried, reflecting the contextual comprehension of the model.

Table11 shows the selected hyperparameters for LoRA. Choosing appropriate hyperparameters in LoRA significantly contributes to achieving such results. The low rank of the adaptation rank 8 is selected to enable the model to capture the task-specific patterns without the introduction of too much computational overhead. The scaling factor alpha was set to 32 so that the low-rank updates of the large models such as Gemma 2B and Open-Llama 3.1B would get amplified. A dropout rate of 0.1 was used to avoid overfitting and thus, generalize well to unseen data. Fine-tuning attention mechanisms are specific modules such as q_proj and v_proj of LoRA, which is a critical module for the context-driven task of QA. A weight decay of 0.01 further aided regularization, preventing overfitting in training.

The performance of LoRA-adapted models for both Malayalam and Tamil shows considerable improvements over their RAG counterparts, especially in semantic alignment and contextual understanding. For Malayalam, the F1-scores for LoRA models like Gemma 2B and Open-Llama 3.1B improved considerably, reaching 0.85 and 0.865 respectively, while cosine similarity metrics showed enhanced semantic closeness, with Open-Llama achieving 0.99. In Tamil, also, the LoRA adaptation raised the F1-score to 0.81 with Open-Llama and 0.73 with Gemma 2B, with respect to the relatively mediocre performance under RAG. The BLEU and ROUGE scores remain low anyway due to the intrinsic limitation of exact-match-based metrics in morphologically rich languages, but in fact there are minor gains with LoRA, with models for Tamil reflecting slightly better BLEU and ROUGE-L values, with 0.01 BLEU and 0.02 ROUGE-L with Open-Llama. These gains can be attributed to targeted fine-tuning of LoRA, which better adapts the model to capture task-specific nu-ances by focusing on low-rank adaptations of critical modules like q_proj and v_proj.

Hyperparameter	**Value**
Rank(r)	8
Scaling Factor (Alpha)	32
Dropout	0.1
Target Modules	q_proj, v_proj
Learning Rate	1e-4
Batch Size	1
Epochs	3

Table11. LoRA Hyperparameters

More importantly, LoRA's attention on contextually relevant embeddings helped it move away from relying on strict matches that allowed for semantically meaningful responses. For example, comparing the results with the RAG baseline, both Malayalam and Tamil performance suffered from poor precision and recall; adaptation with LoRA was fairly balanced, showing improvement on all counts, thus effective in managing the low-resource language.

5.2.4 Quantized-Low Rank Adaptation

The QLoRA fine-tuning approach shows clear improvements over the standard LoRA results, particularly in terms of semantic metrics such as Precision, Recall, and F1-score, for both Malayalam and Tamil. These results indicate that QLoRA is better suitable for tasks involving low-resource languages with complex linguistic structures.

For Malayalam, QLoRA has significantly improved the Recall and F1-score, and Open-Llama achieved a perfect cosine similarity of 1.0, which indicates excellent semantic alignment between generated responses and ground-truth answers. The F1-score for Open-Llama reached 0.8549, which is indicative of superior contextual understanding of the model as compared to LoRA. On the other hand, Gemma 2B in QLoRA also outperformed its LoRA counterpart, particularly in Precision (0.77) and F1 (0.81), showing better control over response quality while still having meaningful contextual representations.

Model / Metric	Gemma 2B	Open-Llama 3.1B	T5-base
BLEU	0	0.01	0.02
ROUGE1	0	0.03	0
ROUGEL	0	0.02	0
Precision	0.77	0.89	0.0490
Recall	0..86	0.86	0.8561
F1-score	0.81	0.85	-0.4686
Cosine Similarity	0.57	1.0	0

Table12. Results of QLoRA for Malayalam

Model / Metric	Gemma 2B	Open-Llama 3.1B	T5-base
BLEU	0.43	0	0.
ROUGE1	0	0	0
ROUGEL	0	0	0
Precision	0.75	0.91	0.05
Recall	0.89	0.95	-0.87
F1-score	0.80	0.93	-0.48
Cosine Similarity	0.58	1.0	0

Table13. Results of QLoRA for Tamil

For Tamil, the manufactured results match the pattern that was found for Malayalam in that QLoRA significantly improves performance in all four metrics. Open-Llama stands out as the strongest model with an F1-score of 0.93 and a cosine similarity of 1.0, meaning better semantic comprehension. Gemmas 2B also exhibits significant improvements with an F1-score of 0.80 and BLEU score of 0.43, suggesting better fluency and closer alignment with reference responses compared to LoRA. These improvements are evident given the intrinsic difficulties in computing exact-match metrics such as BLEU and ROUGE for morphologically dense languages like Tamil.

The hyperparameters used during QLoRA fine-tuning are important for optimizing the training process while addressing the memory and computational constraints related with large models. The 8-bit quantization in QLoRA was chosen to drastically reduce memory requirements and improve computational efficiency so that large models like Gemma 2B and Open-Llama 3.1B can be fine-tuned on resource-constrained hardware without significantly compromising performance. This approach ensures accessibility for researchers working on low-resource languages, making cutting-edge techniques feasible even with limited computational resources.

Hyperparameter	Value
Quantization	8-bit
Rank(*r*)	8
Scaling Factor (Alpha)	32
Dropout	0.1
Target Modules	q_proj, v_proj
Learning Rate	1e-4
Batch Size	1
Epochs	3

Table14. QLoRA Hyperparameters

The choice of these hyperparameters was motivated by the need to balance performance, efficiency, and computational constraints. The rank (r) was set to 8, which allows for low-rank adaptations that effectively capture task-specific knowledge without significantly increasing memory overhead. The scaling factor, or LoRA alpha, was set to 32, providing a balance between the magnitude of parameter updates and model stability. Dropout of 0.1 was used to prevent overfitting, especially because for low-resource languages it degrades generalization more easily. Target modules choice: q_proj and v_proj were chosen to specifically fine-tune the essential layers responsible for query and value projection in an efficient way while letting the core knowledge of the model remain intact.

Batch size was artificially set to 1 in both training and evaluation for large model memory constrains. It was possible that at a low batch size the learning rate of 1e-4 was also sufficient to lead to steady convergence without gradient explosion. Besides, a weight decay of 0.01 could be used in regularization by preventing overfitting of the model in training and logging and evaluation steps being frequent enough to observe performance trend and retain the best model.

This brings us to the introduction of quantization-aware adaptation, with a transition from LoRA to QLoRA, which significantly boosts model performance by reducing the memory requirement and training at lower precision. The procedure allows QLoRA to better leverage the underlying model's capacity for context-rich tasks such as question answering. Compared to LoRA, QLoRA achieves more substantial improvements in semantic metrics across both languages, with F1-scores and cosine similarity. This demonstrates that the model is better adapted to capturing contextual nuances in questions and answers.

5.2.5 Model Reduction

The sizes of the original models were significantly reduced after applying QLoRA (Quantized Low-Rank Adaptation) to the models. QLoRA compresses the model by using a combination of quantization and low-rank adaptation techniques without severely affecting performance. The precision of the model's weights is reduced, for example, using lower-bit quantization, and efficient compression methods are applied to drastically minimize the storage requirements of the models.

Model	Original Size	Reduced Size	Reduction Factor
Open LLaMa 3.1B	12 GB	11.9 MB	~1,008x
Gemma 2B	9.56 GB	8.9 MB	~1,000x
T5-base	0.85 GB	1 MB	~1,000x

Table15. Model Size Reduction after QLoRA

The reduction of model size achieved by QLoRA is very impressive, with substantial compression across different models. Open LLaMa 3.1B originally was 12 GB in size but was compressed down to just 11.9 MB, bringing the reduction factor to about 1,008x. Similarly, Gemma 2B, which originally had a size of 9.56 GB, was compressed down to about 8.9 MB, giving a reduction factor of about 1,000x. The reductions of T5-base from an original size of 0.85 GB to as small as 1 MB lead to the reduction by 1,000x in size; however, this also does not degrade its core capabilities while making them highly efficient to use with storage and computational resources that have very limited space.

6. Conclusion

In this chapter, we discussed the possibility of using miniature versions of Large Language Models (LLMs) to carry out Question Answering tasks for the Dravidian languages with a focus on low-resource languages. Our goal here was to size down the LLM while retaining the performance of the system and allowing it to deploy at edge devices with resource constraints. To this end, we fine-tuned the Open LLaMA model using QLoRA, a methodology that combines low-bit quantization and model compression techniques. This brought the model size down to a mere 11.9 MB, a drastic reduction from its original size, without affecting its ability to handle complex QA tasks in Dravidian languages.

With this approach, we were able to show that QLoRA can efficiently compress large models with retention of their functionality and be suitable for deployment on low resource devices such as mobile phones and IoT devices. In addition, integration with RAG further enhances the generation capabilities of the model, taking into account knowledge outside of the model for the contextual accuracy of its response and thus overall QA performance. This work opens up the potential for deploying tiny LLMs in real-time applications in linguistically diverse environments, especially in Dravidian-speaking regions, to pave the way for the availability of accessible AI-driven solutions on edge devices.

Looking forward, future research should focus on expanding the data set and incorporating a wider variety of questions and topics. Additionally, experimenting with other language models and different optimization techniques could improve further model performance as well as size efficiency. Testing the model's real application on other edge devices should also be done in order to show its practicality and robustness. Ultimately, this work demonstrates that with efficient fine-tuning techniques such as QLoRA, large language models can be adapted for deployment in low-resource settings, thereby scaling NLP technologies for diverse languages and applications.

References

[1] Bibin, P.A., & Babu Anto, P. (2024). *Malayalam Question Answering System Based on a Deep Learning Hybrid Model of CNN and Bi-LSTM Approach*. International Journal of Modern Education and Computer Science.

[2] Rahmath K, R., & Reghu Raj, P.C. (2023). *Malayalam Question Answering System Using Deep Learning Approaches*. IETE Journal of Research.

[3] Liji, S.K., & Ilyas, P.M. (2024). *Neural Word Embedding-Based Transformer Model for Improving Malayalam Question Answering on Health Domain*. International Journal of Intelligent Systems and Applications in Engineering.

[4] Manghat, S., & Schultz, T. (2023). *Data Augmentation Strategies for Low Resource Conversational Code-Switching*. O-COCOSDA Proceedings.

[5] Hu, X., & Xu, X. (2020). *Data Augmentation for Code-Switch Language Modeling by Fusing Multiple Text Generation Methods*. INTERSPEECH Proceedings.

[6] Antony, B., & Paul, N.R. (2023). Question Answering System for Tamil Using Deep Learning. *Communications in Computer and Information Science*.

[7] Krishnan, A., Sriram, S.R., Ganesan, B.V.R., & Sridhar, S. (2023). An Extractive Question Answering System for the Tamil Language. *Advances in Science and Technology*.

[8] Kumar, L.A., Renuka, D.K., & Shunmugapriya, M.C. (2022). Tamil question answering system using machine learning. *Deep Learning Research Applications for Natural Language Processing*.

[9] Murugathas, R., & Thayasivam, U. (2022). Domain specific Question & Answer generation in Tamil. *2022 International Conference on Asian Language Processing (IALP 2022)*.

[10] Bhattu, S.N., Krishna, N.S., & Somayajulu, D.V.L.N. (2018). Idrbt-team-a@IECSIL-FIRE-2018: Named Entity Recognition of Indian languages using Bi-LSTM. *CEUR Workshop Proceedings*.

[11] Holtermann, C., Röttger, P., Dill, T., & Lauscher, A. (2024). Evaluating the Elementary Multilingual Capabilities of Large Language Models with MULTIQ. *Proceedings of the Annual Meeting of the Association for Computational Linguistics*.

[12] Vyawahare, A., Tangsali, R., Mandke, A., et al. (2022). Neural Machine Translation On Dravidian Languages. *DravidianLangTech 2022*.

[13] Chakravarthi, B.R., Priyadharshini, R., Subalalitha, C.N., et al. (2022). Findings of the Shared Task on Multi-task Learning in Dravidian Languages. *DravidianLangTech 2022*.

[14] Priyadharshini, R., Chakravarthi, B.R., Thavareesan, S., et al. (2021). Overview of the

DravidianCodeMix 2021 Shared Task on Sentiment Detection in Tamil, Malayalam, and Kannada. *ACM International Conference Proceeding Series*.

[15] Vasantharajan, C., & Thayasivam, U. (2021). Offensive language identification in Dravidian code-mixed YouTube Comments and Posts. *DravidianLangTech 2021*.

[16] Krishnan, A., Sriram, S.R., Ganesan, B.V.R., et al. (2023). An Extractive Question Answering System for the Tamil Language. *Advances in Science and Technology*.

[17] Liji, S.K., & Ilyas, P.M. (2024). Neural Word Embedding-Based Transformer Model for Improving Tamil Question Answering on Health Domain. *International Journal of Intelligent Systems and Applications in Engineering*.

[18] Vasantharajan, C., Thayasivam, U. *"Hypers@DravidianLangTech-EACL2021: Offensive language identification in Dravidian code-mixed YouTube Comments and Posts"*. Proceedings of the 1st Workshop on Speech and Language Technologies for Dravidian Languages, DravidianLangTech 2021 at 16th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2021.

[19] Saraswathi, S., Geetha, T.V. *"Comparison of performance of enhanced morpheme-based language model with different word-based language models for improving the performance of Tamil speech recognition system"*. ACM Transactions on Asian Language Information Processing, 2007.

[20] Prajapati, R., Parikh, V., Majumder, P. *"IRLAB-DAIICT@DravidianLangTech-EACL2021: Neural Machine Translation"*. Proceedings of the 1st Workshop on Speech and Language Technologies for Dravidian Languages, DravidianLangTech 2021 at 16th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2021.

[21] Devi, S.L. *"Resolving pronouns for a resource-poor language, Malayalam using resource-rich language, Tamil"*. International Conference Recent Advances in Natural Language Processing, RANLP, 2019.

[22] Chanda, S., Pal, S. *"IRLab@IITBHU@Dravidian-CodeMix-FIRE2020: Sentiment analysis for Dravidian languages in code-mixed text"*. CEUR Workshop Proceedings, 2020.

[23] Chakravarthi, B.R., Priyadharshini, R., Banerjee, S., Johnson, M. *"Findings of the Shared Task on Machine Translation in Dravidian languages"*. Proceedings of the 1st Workshop on Speech and Language Technologies for Dravidian Languages, DravidianLangTech 2021 at 16th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2021.

[24] Vyawahare, A., Tangsali, R., Mandke, A., Kadam, D. *"PICT@DravidianLangTech-ACL2022: Neural Machine Translation On Dravidian Languages"*. DravidianLangTech 2022 - 2nd Workshop on Speech and Language Technologies for Dravidian Languages, Proceedings of the Workshop, 2022.

[25] Rahman, T., Raihan, A.B.S., Tanvir Rahman, Md., Hoque, M.M. *"CUET_DUO@DravidianLangTech EACL2024: Fake News Classification Using Malayalam-BERT"*. DravidianLangTech 2024 - 4th Workshop on Speech, Vision, and Language Technologies for Dravidian Languages, Proceedings of the Workshop, 2024.

[26] Zhang, Tianyi, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. "Bertscore: Evaluating text generation with bert." *arXiv preprint arXiv:1904.09675* (2019).